

Software Architecture and Design Specification

Project: File Compression Tool (Basic ZIP Implementation)

Version: 1.0

Authors: Team 14

Date: 03-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	03-10-2026	Team 14	Initial version of Software Architecture and Design Specification

Approvals

Role	Name	Signature/Date
Team Member	Pavan Kumar H K	
Team Member	Pranavi Sistla	
Team Member	Rachit Upadhyay	
Team Member	Inchara K Kuppal	

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the File Compression Tool (Basic ZIP Implementation). The system uses Huffman coding to compress files and provides the corresponding decompression functionality. This document describes the major software components, their interactions, architectural pattern, design decisions, security considerations, interfaces, and error-handling mechanisms of the system.

1.2 Scope

The File Compression Tool is a C/C++ based application that provides basic file compression and decompression using the Huffman coding technique. The system accepts a file as input, analyzes the frequency of its data, constructs a Huffman tree, generates Huffman codes, and produces a compressed output file. It also supports decompression of valid compressed files to reconstruct the original file.

The scope of this design includes the major software components, their interactions, the selected architecture pattern, security considerations, interfaces, error handling, and the design of the compression and decompression workflows. The project focuses on basic lossless file compression using Huffman coding as specified in the assigned problem statement.

1.3 Audience

This document is intended for the following stakeholders:

- **Project Developers:** To understand the system architecture, components, interfaces, and design decisions required for implementation.
- **Testers:** To understand the expected component interactions and system workflows for preparing and executing tests.
- **Project Team Members:** To maintain consistency between the software requirements, architecture, and detailed design.
- **Faculty/Evaluators:** To review the architectural and design decisions of the proposed File Compression Tool.
- **Future Maintainers:** To understand the organization and interactions of the system components for future maintenance and enhancement.

1.4 Definitions

The following terms and abbreviations are used in this document:

- **SAD:** Software Architecture and Design Specification.
- **SRS:** Software Requirements Specification.
- **Huffman Coding:** A lossless data compression technique used to represent frequently occurring symbols with shorter binary codes.
- **Huffman Tree:** A binary tree constructed from symbol frequencies and used to generate Huffman codes.

- **Compression:** The process of reducing the size of a file by encoding its data using Huffman codes.
- **Decompression:** The process of reconstructing the original file from compressed data.
- **Encoding:** The process of converting input data into its corresponding Huffman binary representation.
- **Decoding:** The process of converting Huffman-encoded data back into the original data.
- **CLI:** Command-Line Interface used to interact with the application.
- **FR:** Functional Requirement.
- **NFR:** Non-Functional Requirement.
- **RTM:** Requirements Traceability Matrix.

2. Document Overview

2.1 How to use this document

This document provides the architectural and detailed design specifications for the File Compression Tool. It describes the system architecture, major components, component interactions, selected architecture pattern, technology considerations, security architecture, requirement traceability, and detailed design of the system. The document should be used by the development team as a reference during implementation and by testers and evaluators to understand the structure and behaviour of the proposed system.

2.2 Related Documents

The following documents are related to this Software Architecture and Design Specification:

1. **Software Requirements Specification (SRS)** — Defines the functional and non-functional requirements of the File Compression Tool.
2. **Mini-Project Problem Statement** — Defines the assigned File Compression Tool problem and specifies Huffman coding as the compression technique. Problem Statements-Team-F-Sec
3. **Project Guidelines** — Defines the project proposal, team responsibilities, project description, and product ownership requirements. Project Guidelines
4. **Software Engineering Mini-Project Deliverables Part-1** — Defines the required architecture and design deliverables, including the component diagram,

architecture pattern, security architecture, requirement traceability, sequence diagrams, API design, and error handling.

3. Architecture

3.1 Goals & Constraints

Goals:

- Provide a lossless file compression mechanism using Huffman coding.
- Provide decompression functionality to reconstruct the original file from valid compressed data.
- Maintain the integrity of the original file during compression and decompression.
- Provide a modular architecture that separates file handling, Huffman processing, compression, and decompression responsibilities.
- Provide clear feedback to the user regarding successful operations and errors.

Constraints:

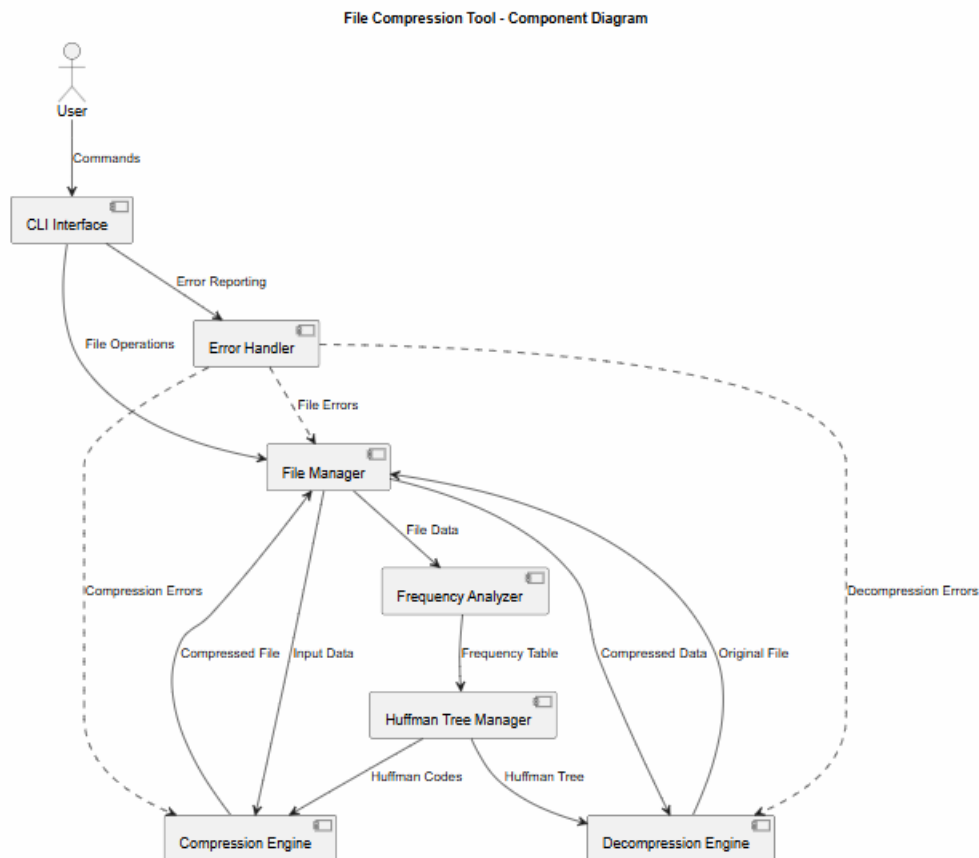
- The system shall be implemented using **C/C++**, as specified in the assigned problem statement. Problem Statements-Team-F-Sec
- The compression technique shall be based on **Huffman coding**.
- The system is intended as a basic file compression tool and does not aim to implement the complete feature set of commercial ZIP utilities.
- The system must handle binary file data appropriately so that the decompressed output can reproduce the original file.
- The architecture and design must remain sufficiently modular to support implementation and testing of individual components.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Project Users	Easy file selection, successful compression/decompression, clear operation status and error messages.

Stakeholder	Primary Concerns
Project Developers	Modular design, correct implementation of Huffman coding, maintainability, and clear component responsibilities.
Testers	Verifiable component behaviour, correct compression/decompression, error handling, and requirement traceability.
Faculty/Evaluators	Compliance with the assigned problem statement, appropriate architecture and design, and clear documentation.
Future Maintainers	Understandable architecture, separation of responsibilities, extensibility, and maintainability.

3.3 Component (UML) Diagram



3.4 Component Descriptions

- **CLI Interface:** Provides the interaction between the user and the File Compression Tool. It accepts commands, file paths, and operation selections for compression and decompression, and displays status and error messages.
- **File Manager:** Handles reading input files and writing compressed or decompressed output files. It provides file data to the processing components and manages file-related operations.
- **Frequency Analyzer:** Reads the input file data and calculates the frequency of each byte/symbol. The resulting frequency information is provided to the Huffman Tree Manager.
- **Huffman Tree Manager:** Constructs the Huffman tree from the frequency information and generates the corresponding Huffman codes required for encoding and decoding.
- **Compression Engine:** Uses the input file data and Huffman codes to encode the data and generate the compressed representation.
- **Decompression Engine:** Uses the compressed data and Huffman tree information to decode the data and reconstruct the original file contents.
- **Error Handler:** Handles errors occurring during file operations, compression, and decompression. It provides appropriate error information to the CLI Interface for display to the user.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture is chosen for the File Compression Tool because it provides clear separation of responsibilities between the user interface, file management, Huffman processing, compression/decompression, and error handling components.

The architecture separates the system into logical layers so that individual components can be developed, tested, maintained, and modified independently.

The major logical layers are:

- **Presentation Layer:** CLI Interface
- **File Management Layer:** File Manager
- **Huffman Processing Layer:** Frequency Analyzer and Huffman Tree Manager
- **Compression/Decompression Layer:** Compression Engine and Decompression Engine

- **Error Handling:** Error Handler

A microservices architecture is not considered necessary because the project is a standalone basic file compression application rather than a distributed system. The layered approach is therefore suitable for maintaining modularity without introducing unnecessary architectural complexity.

3.6 Technology Stack & Data Stores

Programming Language: C/C++

Development Environment: Standard C/C++ development environment with a compiler supporting the required language features.

Core Algorithm: Huffman Coding for lossless data compression.

File Handling: C/C++ file I/O facilities for reading input files and writing compressed/decompressed output files.

Data Structures: Binary tree structures for the Huffman tree and appropriate data structures for frequency analysis and code generation.

User Interface: Command-Line Interface (CLI).

Data Storage: The system uses local files for storing the original input file, compressed output file, and decompressed output file. A relational database is not required for the basic implementation.

Design and Documentation Tools: PlantUML/draw.io may be used for UML architecture and design diagrams.

The assigned problem statement specifies **C/C++** and a **basic ZIP implementation using Huffman coding**.

3.7 Risks & Mitigations

Risk	Mitigation
Incorrect Huffman tree construction	Validate frequency calculation and tree construction using known test inputs before performing compression.
Loss of data during compression/decompression	Verify that the decompressed file matches the original file byte-for-byte.

Risk	Mitigation
Incorrect handling of binary data	Use binary file I/O and appropriate byte-level processing.
Corrupted compressed file	Validate compressed-file metadata and detect invalid or incomplete compressed data before producing the final output.
Invalid input file/path	Validate file existence and accessibility before starting compression or decompression.
Large or unusual input files	Use appropriate data structures and file streaming/processing techniques to avoid unnecessary memory usage.
Component integration errors	Test interactions between File Manager, Huffman Tree Manager, Compression Engine, and Decompression Engine independently and together.

3.8 Traceability to Requirements

Requirement	Requirement Description	Architecture Reference
FR-01	The system shall allow the user to specify an input file for compression.	CLI Interface, File Manager
FR-02	The system shall calculate the frequency of each byte/symbol in the input file.	Frequency Analyzer
FR-03	The system shall construct a Huffman tree using the calculated frequencies.	Huffman Tree Manager
FR-04	The system shall generate Huffman codes from the Huffman tree.	Huffman Tree Manager
FR-05	The system shall encode input data and generate a compressed file.	Compression Engine, File Manager
FR-06	The system shall decompress a valid compressed file and reconstruct the original file.	Decompression Engine, File Manager

Requirement	Requirement Description	Architecture Reference
FR-07	The system shall provide compression information to the user.	CLI Interface, File Manager
FR-08	The system shall handle invalid or inaccessible input files.	File Manager, Error Handler
FR-09	The system shall detect invalid or corrupted compressed files.	Decompression Engine, Error Handler
FR-10	The system shall handle an empty input file without unexpected termination.	File Manager, Compression Engine
NFR-01	The system shall provide acceptable processing performance for supported files.	Compression Engine, Decompression Engine
NFR-02	The decompressed output shall preserve the contents of the original file.	Compression Engine, Decompression Engine
NFR-03	The system shall provide clear user feedback and error messages.	CLI Interface, Error Handler
NFR-04	The system shall maintain modular separation between major processing components.	Overall Layered Architecture

3.9 Security Architecture

The security architecture focuses on protecting the integrity of the compression and decompression processes and preventing invalid input from producing incorrect output.

Security Objectives

1. **Maintain file integrity:** The system shall ensure that valid decompression reconstructs the original file contents without unintended modification.
2. **Protect the decompression process:** The system shall validate compressed-file information and handle malformed or corrupted input safely.

Security Measures

- **Input Validation:** The File Manager shall validate input file paths and file accessibility before processing.

- **Compressed File Validation:** The Decompression Engine shall validate the structure and required metadata of a compressed file before decoding.
- **Error Isolation:** Invalid input shall be handled by the Error Handler rather than causing unexpected termination of the application.
- **Output Protection:** The system shall avoid treating invalid or incomplete decoded data as a successful decompression result.
- **Data Integrity Verification:** The correctness of decompression shall be verified by comparing the resulting file with the original input during testing.
- **No Sensitive Logging:** Error messages and logs, if implemented, shall avoid unnecessarily exposing file contents or other sensitive information.

4. Design

4.1 Design Overview

The File Compression Tool is designed using a modular layered structure to separate user interaction, file management, Huffman processing, compression, decompression, and error handling responsibilities.

During compression, the system receives an input file through the CLI Interface, reads the file through the File Manager, analyzes byte frequencies using the Frequency Analyzer, constructs the Huffman tree and codes using the Huffman Tree Manager, and encodes the input using the Compression Engine. The resulting compressed data is stored as an output file through the File Manager.

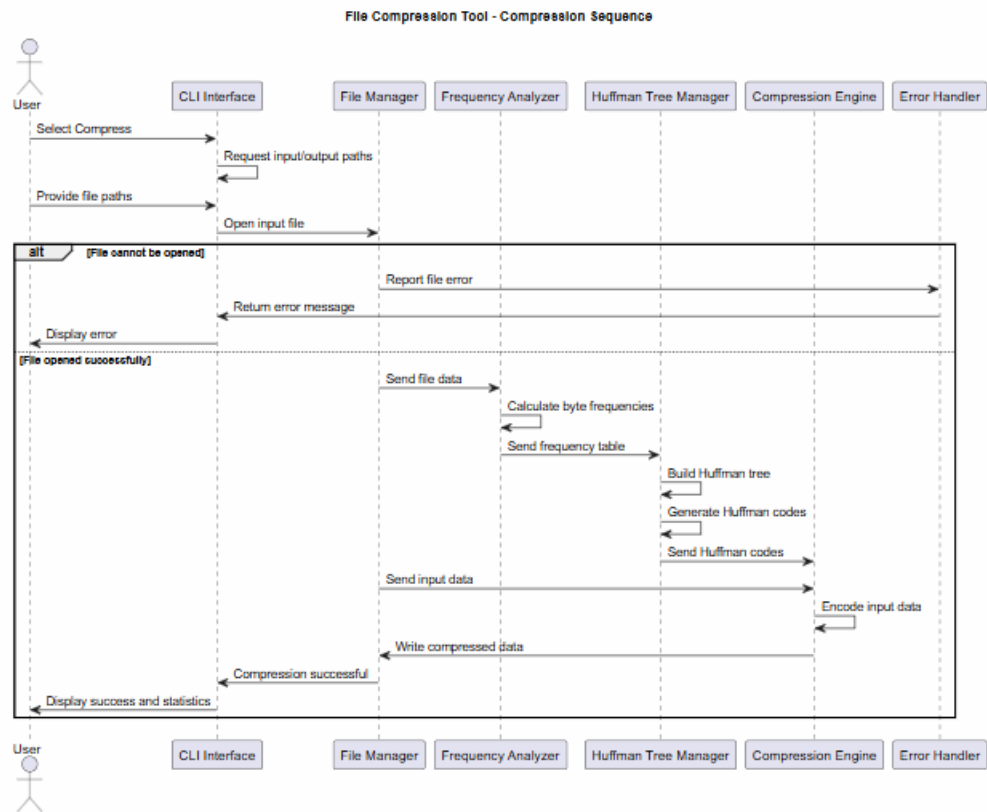
During decompression, the system reads the compressed file, obtains the required Huffman information, decodes the compressed data using the Decompression Engine, and reconstructs the original file. The Error Handler manages invalid inputs, file errors, and processing errors throughout these operations.

The design emphasizes modularity, separation of concerns, file integrity, and maintainability.

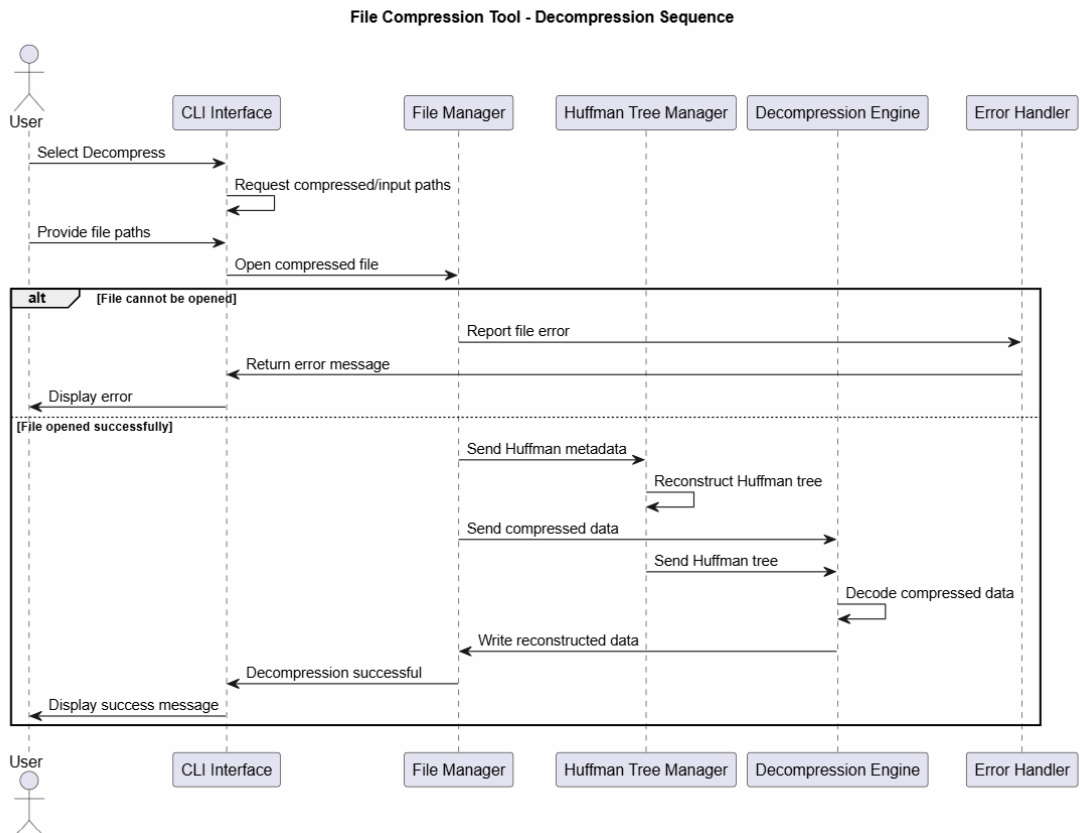
4.2 UML Sequence Diagrams

The following sequence diagrams describe the two primary workflows of the File Compression Tool:

1. File Compression



2. File Decompression



4.3 API Design

The File Compression Tool is primarily a command-line application and therefore does not require a web-based REST API. Instead, the system uses internal component interfaces between the major modules.

Interface 1 — File Manager Interface

Purpose: Provides file reading and writing operations to other components.

Operation Input

Output

readFile() Input file path

File data

writeFile() Output file path, encoded data

Success/failure status

openFile() File path, mode

File handle/status

The File Manager is responsible for file access and ensures that file-related errors are reported to the Error Handler.

Interface 2 — Huffman Tree Manager Interface

Purpose: Provides Huffman frequency analysis and code/tree operations.

Operation	Input	Output
buildTree()	Frequency table	Huffman tree
generateCodes()	Huffman tree	Huffman code table
reconstructTree()	Stored Huffman metadata	Huffman tree

Interface 3 — Compression Engine Interface

Purpose: Encodes input data using Huffman codes.

Operation	Input	Output
compress()	Input data, Huffman codes	Encoded data
calculateStatistics()	Original and compressed sizes	Compression statistics

Interface 4 — Decompression Engine Interface

Purpose: Decodes compressed data using the Huffman tree.

Operation	Input	Output
decompress()	Encoded data, Huffman tree	Original data
validateData()	Compressed data	Validation status

These interfaces provide clear boundaries between the major components and support the modular architecture defined in Section 3.

4.4 Error Handling, Logging & Monitoring

The File Compression Tool uses structured error handling to manage invalid inputs, file access failures, corrupted compressed files, and processing errors.

Error Handling:

- Invalid or inaccessible input files shall be detected before processing.
- Invalid output paths or file-writing failures shall be reported to the user.
- Empty input files shall be handled without unexpected termination.

- Corrupted or incomplete compressed files shall be detected during decompression.
- Invalid compressed-file metadata shall be rejected before decoding.
- Compression and decompression failures shall be reported through the Error Handler to the CLI Interface.

Logging:

The system may maintain basic logs containing the operation type, success or failure status, and relevant error information. File contents and sensitive information shall not be unnecessarily included in logs.

Monitoring:

The system may monitor operation status, processing time, original file size, compressed file size, and compression ratio. These details can be displayed to the user after a successful compression operation.

4.5 UX Design

The File Compression Tool uses a command-line interface designed to provide simple and clear interaction.

The main menu may provide options such as:

```
=====
FILE COMPRESSION TOOL
=====
```

1. Compress File
2. Decompress File
3. Exit

Enter your choice:

For compression, the system prompts the user for the input and output file paths.

Example:

Enter input file: sample.txt

Enter output file: sample.huff

Compressing...

Compression successful.

Original Size : 120 KB

Compressed Size : 72 KB

Compression Ratio: 60.00%

Space Saved : 40.00%

For decompression:

Enter compressed file: sample.huff

Enter output file: restored.txt

Decompressing...

Decompression successful.

Original file restored successfully.

The interface shall:

- Provide clear operation choices.
- Use understandable prompts.
- Display success and failure messages.
- Avoid unnecessary technical information in normal user interactions.
- Inform the user when an invalid input is provided.
- Clearly indicate when an operation has completed.

4.6 Open Issues & Next Steps

Open Issues

- The exact compressed-file header/metadata format may be refined during implementation.
- Handling of special cases such as empty files and files containing only one unique byte requires implementation-level validation.
- Performance for very large files will need to be evaluated during implementation and testing.
- The final command-line interface may be refined based on usability testing.

Next Steps

1. Implement the File Manager component.
2. Implement frequency analysis.
3. Implement Huffman tree construction and code generation.
4. Implement the Compression Engine.
5. Implement the Decompression Engine.
6. Implement compressed-file metadata handling.
7. Implement Error Handler functionality.
8. Integrate all components through the defined interfaces.
9. Perform component-level and integration testing.
10. Validate that decompression reproduces the original file correctly.

5. Appendices

5.1 Glossary:

Term	Definition
Compression	The process of reducing the size of data while preserving the information contained in the original file.
Decompression	The process of reconstructing the original data from compressed data.

Term	Definition
Huffman Coding	A lossless compression technique that assigns shorter binary codes to frequently occurring symbols and longer codes to less frequent symbols.
Huffman Tree	A binary tree constructed from symbol frequencies and used to generate Huffman codes.
Huffman Code	A variable-length binary code assigned to a symbol based on its frequency in the input data.
Frequency Table	A table containing the occurrence frequency of each byte or symbol in the input file.
Encoding	The process of converting input data into its corresponding Huffman-coded representation.
Decoding	The process of converting Huffman-coded data back into the original data.
Compression Ratio	A measure comparing the compressed file size with the original file size.
CLI	Command-Line Interface used by the user to interact with the application.
SAD	Software Architecture and Design Specification.
SRS	Software Requirements Specification.
RTM	Requirements Traceability Matrix.
FR	Functional Requirement.
NFR	Non-Functional Requirement.

5.3 Tools:

- **C/C++ Compiler** — implementation of the File Compression Tool.
- **PlantUML** — creation of UML component and sequence diagrams.
- **Microsoft Word** — preparation of the SAD document.

- **Git/GitHub** — source-code version control and project repository management.